Star Tracker ML Cell-Classification Architecture

Summary with Glossary

1. Project Goal

Build a machine-learning pipeline for an onboard CubeSat star tracker (running on a Raspberry Pi 5) that solves the "lost-in-space" problem more efficiently. The ML step does not compute exact attitude; it produces a coarse pointing prior (which cell or region of sky), then hands off to a traditional star tracker plus Extended Kalman Filter (EKF) for precise attitude determination.
Why: Traditional lost-in-space pattern matching (PMT) must search the entire Hipparcos catalog and compute all pairwise angles, which is very expensive. A coarse cell prior dramatically shrinks the search space, saving compute.
The sky is partitioned into **529 cells** (GLAS-document convention).

2. Overall Pipeline

1. Lost-in-space condition
2. Onboard camera image
3. ML classifier (trained on ground, inference only in orbit)
4. Shortlist of candidate cells (multi-label)
5. Select cell(s) as pointing prior
6. Star tracker and other sensors within an EKF
7. Precise attitude

3. Cell Selection Logic

- **Multi-label output** (sigmoid), not single-cell: several cells appear in the field of view at once.
- **Primary driver:** network confidence (trust the localization).
- **Fallback / tie-breaker:** star richness, meaning which candidate cell is easiest to solve downstream (more bright stars, less noise).
- **Rule:** pick the highest-confidence cell; if that cell is too sparse or noisy to solve, fall back to the richer cell.

4. Key Design Decision: Feature-Based vs. Raw-Image

Two options were considered:

- **Option 1:** a large CNN fed raw images that learns its own features. It must rediscover rotation invariance and needs more data, parameters, and compute. It is a black box and hard to debug.
- **Option 2 (chosen):** a smaller neural network fed hand-crafted rotation-invariant features.

Decision: Option 2, feature-based. Rationale rests on three pillars:

1. More data-efficient and faster to converge, since rotation invariance is baked in mathematically, for free.
2. Explainable and inspectable, so features can be reviewed to see where things went wrong. This is critical for flight software.
3. Lower, predictable, low-power compute on the embedded Raspberry Pi.

Training/inference asymmetry: training is expensive and done on the ground (big GPU, frozen weights). Inference in orbit is a cheap forward pass. Hand-crafted feature computation adds a fixed, predictable, non-neural cost on the Pi, still far cheaper than a bloated raw-image CNN.

5. The Feature: Pairwise Angular Distance Histogram

Inspired by Rijlaarsdam's early ML star-tracking work (pole star plus histogram of distances), but adapted from single-star identification to whole-cell classification, using the full set of stars rather than one pole star.

Extraction steps:

1. **Detect stars (centroiding).** Threshold the background, group bright pixels into blobs, and compute sub-pixel centers, producing a list of positions and brightnesses.
2. **Compute pairwise angular distances.** Convert each star's pixel position to a direction vector via the camera model (focal length, sensor geometry), then take the angle between each pair of vectors. This is rotation-invariant.
3. **Histogram the distances** into bins to form the feature vector.

Important: use angular distances, not pixel distances, to preserve rotation invariance. Be consistent about this with the professor.

Other candidate cell-level features discussed: magnitude histogram, and a radial/angular density map (rings and wedges). The pairwise angular distance distribution is the preferred sweet spot.

6. Binning Strategy (Avoiding Histogram Collisions)

Use roughly 25 bins. Do not divide the distance range uniformly. Use **adaptive (quantile) binning**: place more, finer bins where star-pair distances cluster, near the distribution's mean or median, so that nearby but distinct cells get distinguishable histograms. This gives each cell a more unique fingerprint.

7. "I Don't Know" and Graceful Degradation

Add a confidence threshold with a **rejection option**. When the network cannot confidently pick one cell:

- Do not fall back to searching the whole sky.
- Instead, take every cell above a confidence threshold, union their catalog regions, and hand that shortlist to the pattern matcher.
- One cell when sure; two or three when unsure; the whole sky only in true catastrophic failure.

This is **graceful degradation of the search space**, and it maps directly onto the multi-label output. It still shrinks 529 cells to a handful. Optionally, cascade to a richer second feature only

when the cheap one fails.

8. Sim-to-Real Gap (Synthetic Training Data)

Training data: synthetic star-field images rendered from the Hipparcos catalog. Draw a random attitude quaternion, project stars via the camera model, and label which cells are in frame.
Robustness plan: domain randomization with a physically calibrated noise model.

- Model each real noise source (hot pixels, cosmic ray hits, stray light from the Sun or Moon, sensor read noise, defocus and blur) with its own probability of occurrence and a randomized magnitude distribution, for example a mean effect with a plus/minus spread.
- Anchor those probabilities and magnitudes to real sensor data where possible (dark frames, datasheet read-noise and dark-current numbers) rather than guessing.
- Validate against both clean and noisy test sets (ablation) to prove robustness.

9. Development Stages (High Level)

1. **Data generation** (the heavy lifter): synthetic renderer, realistic noise, and cell labels.
2. **Network architecture:** small CNN or feature-fed net, 529 outputs, sigmoid activation for multi-label.
3. **Training:** iterate to convergence and hold out validation data. Unlimited synthetic data helps avoid overfitting.
4. **Evaluation and quantization:** test on held-out attitudes and convert 32-bit weights to 8-bit for the Pi.
5. **Deployment:** export to TensorFlow Lite and wire into the onboard pipeline as the pointing prior.

10. Validation To-Do Before Committing

Confirm the chosen feature is discriminative: check that different cells produce distinguishable histograms before building the full model.

11. Glossary of Key Terms

Rotation Invariance

Definition: a feature that does not change when the camera rolls or rotates.
In a sentence: "I chose the pairwise angular distance histogram because its rotation invariance means the same patch of sky gives the same feature no matter how the camera is oriented."

Quantile Binning (Adaptive Binning)

Definition: placing histogram bin edges where the data actually clusters instead of spacing them evenly.
In a sentence: "I used quantile binning so that densely packed distances get finer bins, giving each cell a more distinguishable fingerprint."

Confidence Threshold with a Rejection Option

Definition: letting the network say "I don't know" when no cell clears a confidence bar.
In a sentence: "I added a confidence threshold with a rejection option so the system doesn't force a wrong single answer."

Graceful Degradation of the Search Space

Definition: when unsure, union the candidate cell catalogs and search those instead of falling back to the whole sky.
In a sentence: "Through graceful degradation of the search space, an uncertain result still narrows 529 cells down to just two or three."

Domain Randomization (with a Physically Calibrated Noise Model)

Definition: injecting realistic, probability-weighted noise into synthetic training data, with the noise parameters anchored to real sensor characteristics.
In a sentence: "I applied domain randomization with a physically calibrated noise model to close the sim-to-real gap."

Data-Efficient (preferred over "easier to train")

Definition: requiring less training data and a smaller network to converge.
In a sentence: "The feature-based approach is more data-efficient and faster to converge than a raw-image CNN."